SELF-DIRECTED SYSTEMS PROGRAMMING LAB

tiny-kv

Build a Persistent Key-Value Store in Rust

A college-style assignment for learning Rust through a small, correct, durable, and measurable storage engine.

THE LEARNING PATH

Rust foundations / in-memory store / file persistence / append-only log / crash recovery / compaction / profiling and Linux / Mini-LSM

19 sections / 7 assignments / 100-point rubric

Version 1.0 | October 15, 2026

Start with the first-session checklist in section 18. Your first major milestone is file-backed put/get/delete.

Contents

Click a section title or use the PDF bookmarks to navigate.

1. Project brief . . . 3

2. Learning outcomes and project boundaries . . . 4

3. Suggested schedule and working method . . . 5

4. Environment setup . . . 6

5. Rust preparation exercises . . . 7

6. Assignment 1 - In-memory key-value store . . . 8

7. Assignment 2 - File-backed store . . . 9

8. Assignment 3 - Append-only log and acknowledgement . . . 10

9. Assignment 4 - Recovery and failure testing . . . 12

10. Assignment 5 - Compaction and safe replacement . . . 13

11. Assignment 6 - Performance and systems investigation . . . 14

12. Assignment 7 - Bridge to Mini-LSM . . . 15

13. Final storage format and API contract . . . 16

14. Failure test matrix and testing strategy . . . 18

15. Linux debugging and profiling lab . . . 19

16. Assessment rubric and submission checklist . . . 20

17. Engineering workflow and design report . . . 21

18. Your first two sessions . . . 22

19. References and glossary . . . 23

1. Project brief

Project: tiny-kv - Build a Persistent Key-Value Store in Rust
Format: self-directed, college-style programming assignment
Audience: a programmer new to Rust, with older C++ experience
Version: 1.0, October 15, 2026

The central question

What must happen between a caller saying "save this value" and a database honestly replying "saved"?

You will answer that question by building a small database in stages. Begin with an in-memory map, add files, change the storage layout to an append-only log, make recovery explicit, and measure a real performance bottleneck. Then use that foundation to begin Mini-LSM.

The primary goal is understanding. A small implementation that you can explain, break deliberately, repair, and measure is the intended outcome.

Final product

Your database will support three operations:

- **put(key, value):** insert a key or replace its current value.
- **get(key):** retrieve the current value, or report that the key is absent.
- **delete(key):** make the key absent, including after restarting.

You will also add **compact**, which rewrites the current live data into a smaller log. The final program will use a command-line interface and store its database in a local file.

The project includes a test suite, a written durability contract, a recovery demonstration, CPU and memory observations, a reproducible performance comparison, and a short design report. These are part of the implementation, not optional decorations.

Relationship to your career goal

This project develops foundations relevant to database engineering: systems programming, storage formats, I/O, debugging, failure reasoning, and performance analysis. Storage engines are the initial area of depth. Later work can build toward search internals or distributed systems.

Finishing this assignment is a foundation for further practice and reviewed work. It is not a promise of job readiness or an official requirement from any employer.

Materials already in this repository

The repository contains this handbook, a printable PDF, a minimal Rust orientation program, a progress checklist, an engineering journal template, and a performance-report template. The database functionality is your assignment to implement.

First checkpoint: complete Assignment 2 and demonstrate file-backed put/get/delete across separate program runs. Read the later assignments to understand the direction, then work one checkpoint at a time.

2. Learning outcomes and project boundaries

Learning outcomes

By the end of the core project, you should be able to:

1. Write small Rust modules using ownership, borrowing, structs, enums, `Option`, and `Result`.
2. Explain how an in-memory collection differs from a persistent file representation.
3. Specify a byte format that another person could implement independently.
4. Trace a write through application code, the operating system, and a storage synchronization request.
5. State which writes must survive recovery and which have uncertain outcomes.
6. Detect invalid records and recover a permitted incomplete final write.
7. Explain why a deletion needs a persistent record.
8. Compact a log while preserving the live state and respecting crash boundaries.
9. Write tests that challenge assumptions rather than merely exercise the happy path.
10. Use release builds, latency measurements, and profiles to evaluate a change.
11. Observe file-related system calls on Linux and inspect a failure with a debugger.
12. Explain how this log-based store differs from an LSM-tree storage engine.

Required operating model

The final core design assumes a local POSIX-style filesystem and exactly one process accessing a database at a time. Operations within that process are sequential. Running two commands concurrently against the same file violates the project's operating model unless you implement the optional locking extension.

The authoritative data is the file on disk. An in-memory `HashMap<String, String>` holds the current live state after recovery. The full live dataset must fit in RAM. This is a simplifying design choice with a measurable memory cost.

Keys are nonempty UTF-8 strings. Values are UTF-8 strings and may be empty. Apply the size limits in section 13. A key can be overwritten; only its most recent valid operation determines its visible state. Deleting an absent key succeeds and leaves it absent.

Scope of the core implementation

Implement single-process persistence, recovery, compaction, and measurement. Keep the interface small so that the entire write and recovery paths remain understandable.

An SQL parser, HTTP service, GUI, authentication system, Kubernetes deployment, vector search, transactions, and replication are possible later projects. They are outside this assignment's graded core.

Failure model

You will test normal exits, abrupt process termination, failed writes or synchronization requests, incomplete final records, and deliberately malformed files. Checksums provide corruption detection; they do not restore damaged data or guarantee that every possible corruption is detectable.

The durability contract relies on the operating system, filesystem, and storage hardware honoring successful synchronization requests. Your tests must describe which layer they actually exercise. Killing a process leaves the OS running; it is not a power-loss experiment.

3. Suggested schedule and working method

Planning estimate

Allow roughly **12-16 weeks at 6-10 focused hours per week** as an initial planning budget. This is a flexible estimate, not a deadline. Rust ownership, recovery, and debugging can each take additional time. Full completion of Mini-LSM extends beyond this introductory schedule.

- **Weeks 1-2:** environment setup, targeted Rust reading, and small exercises.
- **Weeks 3-4:** Assignment 1, the in-memory store and command handling.
- **Week 5:** Assignment 2, clean-restart persistence.
- **Weeks 6-7:** Assignment 3, record format and append-only log.
- **Weeks 8-9:** Assignment 4, interrupted writes and recovery.
- **Week 10:** Assignment 5, compaction and replacement ordering.
- **Weeks 11-12:** Assignment 6, workloads, profiles, and Linux exercises.
- **Weeks 13-14:** documentation, demonstrations, and Assignment 7 readiness review.
- **Weeks 15-16:** buffer for repairs and the first Mini-LSM chapter.

Move forward when you meet a checkpoint's acceptance criteria. A calendar week ending is not evidence that an implementation is ready.

A useful study session

For a 60-90 minute session:

1. Choose one behavior or question, such as "what does get return for a missing key?"
2. Spend 10-15 minutes reading only the concepts required for that question.
3. Predict the result of one example or failure case.
4. Implement a small change and run the relevant checks.
5. Compare the result with your prediction.
6. Write a brief journal entry and make a focused commit when appropriate.

If a task feels too large, split it. "Implement recovery" can become "decode one record," "reject an invalid operation," "stop at an incomplete prefix," and "truncate before the next append."

Getting help productively

Bring the smallest useful example, the complete compiler error or test failure, your expected result, and what you have already tried. Ask for an explanation or a hint before asking for a full replacement implementation.

You may use documentation, examples, and AI assistance. Record important borrowed ideas and sources. For any code you keep, be able to explain its inputs, outputs, ownership, error paths, and effect on persistent state. If you cannot explain it, break it down before advancing.

Completion rule for each assignment

Complete the behavior, run the required checks, record evidence, answer the explanation questions, and create the checkpoint tag. Keep later milestones unchecked in `docs/progress.md` until their evidence exists.

4. Environment setup

Required tools

- A terminal and a code editor. VS Code with rust-analyzer is one option.
- Current stable Rust, installed through rustup, including Cargo.
- Git and access to your GitHub repository.
- A Linux environment by Assignment 6: a VM, a remote development machine, or a container suitable for the chosen tools.

You can complete the early assignments on macOS. Python is used only to regenerate this handbook's PDF.

Install and verify Rust

Use the official installation instructions at https://www.rust-lang.org/tools/install. On macOS, if the compiler reports a missing system linker, install Apple's Command Line Tools with `xcode-select --install` and finish the installer.

Open a new terminal and verify:

```
rustc --version
cargo --version
rustup show
```

The starter uses Rust edition 2024 and requires Rust 1.85 or newer. The current stable toolchain is recommended.

From the existing project directory:

```
cd ~/Desktop/Projects/tiny-kv
cargo run
```

Expect the orientation message. This repository already has a Cargo project; there is no need to run `cargo new` inside it.

Essential Cargo commands

```
cargo check
cargo run
cargo test
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo build --release
```

`check` checks compilation quickly. `run` compiles and executes. `test` runs tests. `fmt --check` reports formatting differences; use `cargo fmt` to apply formatting. Clippy provides additional diagnostics. Release builds enable optimizations and are the basis for performance measurements.

If needed, add development components:

```
rustup component add rustfmt clippy
```

Project layout as it grows

Use `src/main.rs` for command parsing and output. Put reusable storage behavior in `src/lib.rs`. Split into `src/store.rs`, `src/record.rs`, and `src/error.rs` when that improves readability. Add integration tests under `tests/`. A workload runner may live in `src/bin/workload.rs`.

Use `data/` for disposable local database files and temporary directories for tests. Use a separate database path for every independent test. The Git ignore rules keep generated data and build products out of commits.

Dependency policy

Start with the standard library. Add a dependency when it teaches or solves a concrete requirement. Assignment 2 can use `serde` and `serde_json`; Assignment 3 can use `crc32fast`; tests can use `tempfile`. A CLI parser such as `clap` is optional after you understand the interface.

Commit `Cargo.lock` once Cargo creates it because this repository builds an application. Record why each dependency exists. Implement the storage logic yourself rather than using an embedded database library to satisfy the assignment.

5. Rust preparation exercises

Read just enough to build

Use The Rust Programming Language as the primary reference. Focus on chapters 1-6, 8-9, and 11: setup, basic syntax, ownership, structs, enums, collections, errors, and tests. Chapter 7 helps when splitting modules. Chapter 12 is useful for CLI and file-I/O examples. Learn traits, iterators, and concurrency more deeply when the project requires them.

Read a small section, then write an example. Rustlings can provide extra exercises when a particular concept remains unclear.

Exercise P1: variables and functions

Write a function that accepts two integers and returns their sum. Call it from `main`. Change an immutable binding and observe the compiler error, then make the intended binding mutable.

Explain: What is a type? What changes when you write `mut`? How does a function return a value?

Exercise P2: owned strings and borrowed strings

Create a `String`, pass it to a function by value, and try using it afterward. Read the compiler's explanation. Then write a function that accepts `&str` and call it with a borrowed string.

Explain: Which function takes ownership? Which borrows? Why can borrowing avoid a copy? When is an explicit `clone` appropriate, and what does it cost?

Exercise P3: a map and an optional result

Begin with this small program:

```
use std::collections::HashMap;

fn main() {
    let mut store = HashMap::new();
    store.insert(String::from("name"), String::from("Alice"));
    println!("{:?}", store.get("name"));
}
```

Predict the result before running it. Look up an absent key. Overwrite `name` with `Bob`. Remove `name`, then look it up again. Use `match` to print a friendly message for `Some` and `None`.

Explain: Why is an empty value different from a missing key? What does the map own? What does `get` borrow?

Exercise P4: structs, enums, and errors

Wrap the map in a `KvStore` struct. Represent a parsed command with an enum such as `Put`, `Get`, `Delete`, or `Quit`. Write a parser returning `Result<Command, ...>` and report an unknown command as an error.

Use `?` in a small function that reads a text file. Deliberately try a missing path and inspect the error. Reserve `unwrap` in application code for situations whose invariant you can justify; expected user and I/O errors should travel through `Result`.

Exercise P5: one meaningful test

Write a test that inserts a key, overwrites it, and checks that the latest value is returned. Add a separate test showing that missing and empty-valued keys are distinct. Deliberately break the overwrite behavior and verify that a test fails.

Readiness check

You are ready for Assignment 1 when you can modify these exercises without copying an entire solution and explain `String`, `&str`, `&self`, `&mut self`, `Option`, and `Result` at a practical level. Looking up syntax is normal.

Submit the exercises as a small practice module, an `examples/` program, or journal snippets. Keep the main database implementation easy to find.

6. Assignment 1 - In-memory key-value store

Objective: define correct behavior before introducing files.
Prerequisites: section 5.
Checkpoint tag: `a1-memory`.

Required behavior

Create a `KvStore` backed by `HashMap<String, String>`. Support put, get, and delete. A repeated put replaces the previous value. Deleting an absent key is an idempotent success. Apply the key and value limits from section 13 so later storage work preserves the same public behavior.

A suggested library interface is:

```
pub fn put(&mut self, key: String, value: String)
    -> Result<(), StoreError>;
pub fn get(&self, key: &str) -> Option<&str>;
pub fn delete(&mut self, key: &str) -> Result<(), StoreError>;
```

These signatures are design guidance, not a complete implementation. Define a useful error type and explain your choices. Borrowing from `get` avoids allocating a new string for every lookup.

Interactive interface

Because this version has no persistence, allow several commands in one running process. Accept `put KEY VALUE`, `get KEY`, `delete KEY`, and `quit` from standard input.

For this first parser, define keys as whitespace-free tokens. For put, treat the remaining text after the key as the value. Document how an empty value is entered; one acceptable rule is that `put key` stores an empty string. The library can support any valid UTF-8 key allowed by section 13 even if the initial parser is more limited.

Keep command parsing separate from map operations. Invalid commands should print a useful message and leave the store unchanged. The program should continue until `quit` or end-of-input.

Implementation steps

1. Write the behavior of missing, existing, overwritten, and deleted keys in plain language.
2. Implement the library operations and validation.
3. Write tests for the state transitions below.
4. Add the command enum and parser.
5. Connect the parser to the library and demonstrate several operations in one session.

Acceptance tests

- A new store returns no value for an unknown key.
- Put followed by get returns the supplied value.
- Two puts to one key expose the second value.
- Deleting one key leaves another key unchanged.
- Deleting twice succeeds and leaves the key absent.
- An empty value is present, not missing.
- An empty or oversized key is rejected without changing state.
- Unicode values round-trip through the library.
- A malformed command does not terminate the interactive session unexpectedly.

Deliverables and explanation questions

Submit the code, tests, and a short terminal transcript. Explain who owns stored strings, why mutation needs `&mut self`, and why `get` returns `Option`. Predict what happens to the data when the program exits, then verify it.

Pass condition: all listed behaviors are demonstrated, errors are handled, and you can trace a command from input to map update.

7. Assignment 2 - File-backed store

Objective: reach the first major milestone: put/get/delete that survive a normal restart.
Prerequisites: Assignment 1 and basic file I/O.
Checkpoint tag: `a2-snapshot`.

Storage design

Save the entire live map as a versioned JSON snapshot after each mutation. A suggested top-level structure contains `version: 1` and an `entries` object. Load it when opening the database.

Use serialization rather than inventing delimiter escaping. `serde` with its derive feature and `serde_json` are appropriate here. Validate loaded entries with the same limits as newly supplied entries.

For this checkpoint, a missing database file means a new empty store. An existing malformed file, an unsupported snapshot version, or an inaccessible file must produce a clear error. Distinguish "file not found" from "permission denied." Do not replace an unreadable database with an empty map.

Move to a process-per-command interface

Implement:

```
cargo run -- --db data/snapshot.json put name Alice
cargo run -- --db data/snapshot.json get name
cargo run -- --db data/snapshot.json put name Bob
cargo run -- --db data/snapshot.json delete name
```

Each command opens the store, performs one operation, and exits. Shell quoting allows values with spaces, for example `put name "Alice Smith"`. Make the database path explicit using `--db`. `get` prints the value to stdout. Successful put/delete print `OK` only after saving succeeds.

Use exit status 0 for success, 1 for a missing get, and 2 for usage, validation, file-format, or I/O errors. Print errors to stderr. An empty stored value prints a newline with exit status 0, so it remains distinguishable from a missing key.

Implementation steps

1. Add `KvStore::open(path)` and load the snapshot.
2. Separate parsing, file access, and storage operations.
3. Save the full snapshot after put and delete.
4. Propagate save failures; the CLI must not print `OK` after an error.
5. Add subprocess tests or a scripted transcript spanning separate invocations.

Acceptance tests

- Insert, exit, restart, and retrieve the value.
- Overwrite, exit, restart, and retrieve the replacement.
- Delete, exit, restart, and confirm absence.
- Round-trip spaces, Unicode, newlines through the library, and empty values.
- Open a missing file as an empty store.
- Reject malformed JSON, unsupported versions, and invalid entries.
- Demonstrate an I/O failure and a nonzero exit status without `OK`.

Required design note

Describe this checkpoint's guarantee as **clean-restart persistence**. Explain the cost of serializing and rewriting the whole map for every change. Explain what could happen if rewriting stops midway and why a completed user-space write alone does not establish power-loss durability.

The final log format is different from this educational snapshot. Preserve this checkpoint in Git and use a new path such as `data/store.kv` for Assignment 3. Automatic migration is optional; wrong-format files must be rejected clearly.

8. Assignment 3 - Append-only log and acknowledgement

Objective: store changes incrementally and define exactly when success is reported.
Prerequisites: Assignment 2, `Read`/`Write`, and section 13.
Checkpoint tag: `a3-log`.

Core design

Replace whole-map rewriting with an append-only operation log. A put record stores a key and its new value. A delete record is a tombstone: a persistent instruction that the key is absent. Rebuild the map by replaying records in order; the newest valid operation wins.

Implement the exact v1 format in section 13. Use a small codec module for encoding and decoding. Rust strings in memory are not an on-disk format; integer sizes, byte order, record boundaries, and checksums must be explicit.

Mutation sequence

For each put or delete:

1. Validate the key and value before writing anything.
2. Encode the complete record in memory.
3. Append all bytes, handling short writes with `write_all` or equivalent logic.
4. Ensure any user-space buffer is flushed to the file.
5. Call `File::sync_all()` and propagate any error.
6. Apply the operation to the in-memory map.
7. Return success; the CLI may now print `OK`.

The required baseline synchronizes every mutation. A later optimization must preserve that contract or explicitly be treated as a different experiment.

Durability contract to document

An acknowledged mutation must be present after recovery under the stated single-writer, filesystem, and storage assumptions. A mutation for which the caller receives an error or no reply has an **uncertain outcome**: it may or may not be recovered.

If append or synchronization fails, do not acknowledge success. Mark the open handle unusable and require a close and reopen before any further operation. A partially appended record could otherwise make subsequent records unreachable during recovery, and an uncertain outcome can make the in-memory state misleading.

A complete unacknowledged record may be replayed. This project has no separate transaction-commit record and does not promise exactly-once delivery.

Initial file creation

For a new database, create the file without clobbering an existing one, write the magic header, synchronize the file, and synchronize its parent directory on the supported POSIX platform before acknowledging mutations. Directory synchronization matters because file contents and the filename's directory entry are distinct persistent state.

Use an existing parent directory. Report an incomplete or invalid existing header as an error rather than assuming an empty database.

Acceptance tests and deliverables

- Encode and decode put/delete records, including empty values and Unicode.
- Compare one encoded record against independently specified expected bytes.
- Replay overwrite/delete/reinsert sequences and obtain the expected final state.
- Confirm that file size grows by records rather than whole snapshots.
- Exercise separate-process put/get/delete commands on the log-backed store.
- Inject an append or sync error and verify that success is withheld.
- Submit a write-path diagram and a one-paragraph durability contract.

Explain `write` versus `write_all`, user-space flushing versus `sync_all`, and why an error does not prove that a write is absent from disk.

9. Assignment 4 - Recovery and failure testing

Objective: define and test every recovery decision.
Prerequisites: Assignment 3 and the failure matrix in section 14.
Checkpoint tag: `a4-recovery`.

Recovery algorithm

Validate the file header, then scan records from the first record offset. Track the byte offset immediately after the last fully validated record. Validate lengths before allocating; verify the checksum and payload fields before applying the operation.

Classify the next input as one of:

- **Clean EOF:** no more bytes remain. Recovery succeeds.
- **Valid complete record:** apply it and advance the last-valid offset.
- **Incomplete final prefix or payload:** discard the incomplete tail under the assignment's interrupted-append model.
- **Complete invalid record:** return a corruption or format error. Do not skip it and continue.
- **Read failure:** return an I/O error. A failed read is not normal EOF.

Before accepting subsequent writes after an incomplete tail, truncate the file to the last-valid offset and synchronize that truncation. Otherwise the next valid append would sit behind broken bytes and could be lost on the next recovery.

Important limitation

A damaged length field can sometimes resemble an interrupted append. Tail repair is a policy under a stated failure model, not proof of the cause. Full-record checksum or semantic failures are hard errors, including at the end of the file. Document this distinction explicitly.

Required tests

1. Begin with a valid header and at least two known good records.
2. Encode one additional final record.
3. Produce a separate file for every byte cut within that final record, including inside its prefix and payload.
4. Reopen each file and verify the valid prefix's state and repaired file length.
5. Append a fresh record after recovery, reopen again, and verify it is visible.
6. Test the fully present final record separately; it should replay normally.

Add malformed-length, checksum, opcode, UTF-8, invalid-header, and unsupported-version cases. Keep parsing semantic tests meaningful by recomputing the checksum when deliberately constructing a well-framed but semantically invalid record.

Process-failure demonstration

Use a child process and explicit failpoints or a parent/child handshake to terminate the writer at known boundaries. Avoid depending on an arbitrary sleep to hit a rare window. Verify that acknowledged writes remain and that unacknowledged writes are treated as uncertain.

Injected I/O failures can use a small writer abstraction capable of failing after a chosen number of bytes or at synchronization. An injected failure tests your error handling; it does not simulate all hardware behavior.

Deliverables

Submit automated failure tests, a short crash demonstration transcript, and a recovery-policy document. Answer: Why is a tombstone necessary? Why not ignore every invalid record? Why repair a tail before appending? Why is `kill -9` insufficient to establish power-loss durability?

10. Assignment 5 - Compaction and safe replacement

Objective: reclaim obsolete records while preserving current state and the durability contract.
Prerequisites: Assignment 4.
Checkpoint tag: `a5-compaction`.

Why compaction exists

Repeated overwrites and deletes make the log grow even if the live map stays small. Compaction writes a replacement containing one put record per live key. An empty store still has a valid file header.

In this one-file, full-rewrite design, deleted keys need no tombstone in the replacement because no older records remain there to resurrect them. This reasoning changes for systems that retain older tables or snapshots.

Required replacement protocol

Under exclusive single-process access:

1. Create a temporary file in the same directory as the database, using a path such as `<db>.compact.tmp` and exclusive creation.
2. Write the v1 header and all current live entries. Sort keys if you want deterministic output.
3. Flush any user-space buffer and synchronize the temporary file.
4. Atomically rename the temporary file over the database path on the supported POSIX filesystem.
5. Synchronize the parent directory so the replacement name is made durable under the stated assumptions.
6. Open the replacement as the active log and retire the old handle.
7. Return success only after the required steps complete.

Rename must stay on the same filesystem. An already-open handle can still refer to the old inode after replacement; appending through it would not update the database now found at the path.

Any error leaves the operation unacknowledged and the current handle unusable until reopen. After restart, open the authoritative database path and validate it.

Temporary-file policy

A leftover temporary file is not automatically committed. On startup, validate the main database first. If it is valid, a stale compaction temporary file can be removed before a new compaction attempt. If the main path is absent or invalid while a temporary file exists, report an explicit error rather than silently choosing a file as authoritative.

Acceptance tests

- Populate many keys, overwrite some repeatedly, and delete others.
- Capture the expected live state and original file size.
- Compact, reopen, and compare every live and deleted key.
- Verify the file shrinks on the overwrite-heavy dataset.

- Compact an empty store and compact twice in succession.
- Append after compaction and verify the new write after another reopen.
- Interrupt before temporary-file sync, after sync, after rename, and after directory sync.
- Verify recovery uses a valid state and preserves previously acknowledged data under the tested model.

Record the difference between process-termination observations and the filesystem reasoning required for a machine crash. Submit the ordering diagram and explain why both file sync and directory sync appear.

11. Assignment 6 - Performance and systems investigation

Objective: improve one measured bottleneck while preserving correctness.
Prerequisites: Assignment 5 and section 15.
Checkpoint tag: `a6-performance`.

Build a workload runner

Use a library-level runner in one process, such as `src/bin/workload.rs`. Repeated `cargo run` commands include build checks, process startup, and database recovery; they are useful end-to-end measurements but poor substitutes for measuring an individual operation.

Make the workload deterministic using a fixed seed or a reproducible key sequence. Record dataset size, key/value sizes, operation mix, hit rate, and whether opening/recovery is timed. Start small enough to iterate quickly, then use a larger dataset to expose a meaningful cost.

Suggested workloads:

- **Read-heavy:** prepopulate keys, then perform known hits and misses.
- **Durable writes:** insert and overwrite keys while synchronizing every mutation.
- **Recovery:** reopen logs with the same live set but different amounts of obsolete history.
- **Compaction:** rewrite an overwrite-heavy log and measure time and disk-space change.

Required baseline

Build in release mode. Warm up where appropriate and run each chosen experiment at least three times. Record operations per second, per-operation p50/p95/p99 where applicable, sample count, file size, recovery time, and peak resident memory.

Use a CPU profile to locate a hot path. When a profile is dominated by waiting for I/O, say so; CPU samples alone do not explain all elapsed time. Correlate with synchronization counts or syscall timing where useful.

Do not claim reliable tail behavior from a tiny sample. Explain the percentile calculation or histogram tool you used, and report variation between runs.

Choose one evidence-based change

Examples include reusing an encoding buffer, avoiding unnecessary string clones, reducing allocations during recovery, improving read buffering, or reserving map capacity when justified.

Write the hypothesis before making the change. Run correctness and recovery checks after it. Repeat the same workload, build mode, durability policy, and environment. An optimization that does not help is still useful evidence if you explain the result.

Batching several writes per synchronization changes acknowledgement behavior unless carefully designed. If studied as an extension, report it as a separate durability policy rather than a free speedup over the required baseline.

Deliverables

- A reproducible workload command and dataset definition.
- A completed `results/performance-report.md`.
- CPU and memory evidence with tool names and units.
- A before/after comparison and explanation of tradeoffs.
- The Linux lab transcript from section 15.

Pass condition: measurements support a specific conclusion, the result can be reproduced, and the change preserves the claimed semantics. There is no required throughput number or improvement percentage.

12. Assignment 7 - Bridge to Mini-LSM

Objective: demonstrate readiness and begin the next storage-engine project.
Prerequisites: Assignments 1-6.
Checkpoint tag: `a7-ready`.

Readiness questions

Answer these in your own words and use your implementation as evidence:

1. What lives in memory, and what lives on disk?
2. How does the newest operation for a key win during recovery?
3. Why is delete represented by a record rather than just removing a map entry?
4. At what exact point may your CLI print `OK`?
5. What can the caller conclude when it receives no reply?
6. Which incomplete bytes can recovery discard, and which errors stop opening?
7. Why must a repaired log be truncated before another append?
8. Why does compaction need a replacement protocol?
9. What limits your store as the live dataset grows beyond RAM?
10. What did profiling reveal that you did not expect?

You may consult documentation for syntax. If the storage reasoning is unclear, return to the relevant test and trace a small example.

Compare with Bitcask

Your baseline keeps all live values in RAM. A Bitcask-style design typically keeps an in-memory index mapping keys to locations of their newest on-disk records and reads values from disk. Both use append-friendly records and need recovery and reclamation, but their memory and read costs differ.

Before Mini-LSM, be able to sketch this offset-index design. As an optional bridge, modify a separate branch to store file offsets and lengths instead of all values. Validate record boundaries and measure the memory/read tradeoff.

Begin Mini-LSM

Read the current Mini-LSM prerequisites and course overview. Follow its current repository setup instructions and test commands; course details can change. Keep the course implementation in a separate repository and link it from this project's final report.

Start with the guided course path. Complete the first implementation chapter, record the chapter name and commit, and explain how its data structures differ from tiny-kv. This handbook's completion requirement is the readiness review and initial chapter, not the entire Mini-LSM curriculum.

Concepts to expect next

- **Memtable:** a sorted in-memory write structure.
- **SST:** an immutable sorted on-disk table.
- **Blocks and indexes:** ways to find data without loading the whole table.
- **Merging iterators:** a unified ordered view across several sources.
- **Compaction:** merging tables while handling overwritten and deleted data.
- **Caching:** exchanging memory for fewer or faster reads.
- **MVCC and concurrency:** maintaining correct views while operations overlap.

After Mini-LSM, choose a focused extension toward replication/consistency labs or search internals. Use the evidence from this project to decide which direction interests you most.

13. Final storage format and API contract

Public behavior

The library accepts UTF-8 strings. Keys must contain 1-65,536 bytes. Values may be empty. Limits are measured in bytes, not characters. The encoded payload may contain at most 1,048,576 bytes, including its nine bytes of metadata. Therefore `key_bytes + value_bytes` must be at most 1,048,567.

Validate with checked arithmetic before converting sizes to `u32`. Invalid input leaves memory and disk unchanged. Empty values are distinct from absent keys. Delete is idempotent and can always append a tombstone, even when the key is already absent.

File header

The file begins with exactly eight ASCII bytes:

```
TKVLOG01
```

There is no newline after the header. The `01` suffix identifies format version 1. A file containing only this complete header is a valid empty database. A zero-byte existing file, partial header, unknown version, or wrong magic is an error. Only a genuinely missing path is eligible for new-file initialization.

Record layout

Each record consists of an eight-byte prefix followed by a payload. All integers use **unsigned 32-bit little-endian** encoding. Offsets below are relative to the beginning of one record:

```
offset  width       field
0       4           payload_len
4       4           checksum
8       1           operation: 1 = put, 2 = delete
9       4           key_len
13      4           value_len
17      key_len     key bytes, UTF-8
17+K    value_len   value bytes, UTF-8
```

The payload begins at record offset 8. Its length must equal `9 + key_len + value_len`. A complete record occupies `8 + payload_len` bytes. The maximum payload is 1 MiB; reject an advertised length outside 9-1,048,576 before allocating its payload buffer.

The checksum is standard CRC-32/ISO-HDLC, also commonly called IEEE CRC-32. `crc32fast` implements this variant; the known input `123456789` has checksum `0xCBF43926`. Store the checksum integer in little-endian form.

Checksum input is the exact four serialized `payload_len` bytes followed immediately by the entire payload. The checksum field itself is excluded. Specifying the length coverage helps detect changes to framing metadata when a complete record is available.

Operation rules

- For put, operation is 1, key length is positive, and value length may be zero.
- For delete, operation is 2, key length is positive, and value length must be zero.
- Unknown operations, invalid UTF-8, inconsistent lengths, trailing bytes inside a payload, and exceeded limits are format errors.
- Replay a put by replacing the map entry; replay a delete by removing it.

Example: put key `a`, value `bc` has payload length 12 and total record length 20. Delete key `a` has payload length 10 and total record length 18. Derive one complete byte fixture, including its checksum, using an independent calculation or reference procedure and keep it in the codec tests.

Recovery boundary rules

Zero bytes available at a record boundary is clean EOF. One to seven bytes remaining is an incomplete final prefix. A complete prefix with a permitted payload length but fewer payload bytes remaining is an incomplete final payload. Under the interrupted-append model, truncate these tails to the previous valid boundary and synchronize before using the file for further writes.

A complete record with an invalid checksum or payload is an error, even if it is last. An out-of-range advertised length is also an error. Do not search ahead for a plausible next record. Do not silently turn an invalid header into an empty database.

Apply a record only after fully validating it. On any hard error, opening fails; do not expose a partially replayed store as though recovery succeeded.

Command-line contract

After Assignment 3, the target interface is:

```
tiny-kv --db PATH put KEY VALUE
tiny-kv --db PATH get KEY
tiny-kv --db PATH delete KEY
tiny-kv --db PATH compact
```

Use `cargo run -- ...` during development or `target/release/tiny-kv` after a release build. Put/delete/compact print `OK` only after their required persistence steps succeed. Get prints the value. Exit status is 0 for success, 1 for a missing get, and 2 for errors. Error details go to stderr. Require exactly the appropriate number of arguments.

Only one process may access a database at a time in the core assignment. The CLI must document this assumption. An optional locking extension must cover the entire open/recovery/operation lifecycle, not just the final append.

14. Failure test matrix and testing strategy

Reference-model tests

Use a plain in-memory map as an independent model of logical behavior. Generate a reproducible sequence of puts, gets, and deletes over a small key set. Compare each result and periodically close/reopen the database. Include overwrites, missing deletes, empty values, and non-ASCII strings.

A reference model checks state transitions. It does not establish durability ordering; those require separate tests and reasoning.

Required failure matrix

- **Clean close and reopen:** all acknowledged writes remain.
- **EOF exactly between records:** recovery succeeds without discarding a valid record.
- **Every cut within the final record:** valid prefix survives, partial tail is removed, and later appends remain reachable.
- **Complete but unacknowledged record:** recovery may include it; the caller had an uncertain outcome.
- **Full-record checksum mismatch:** open fails without silently accepting a prefix as the complete database.
- **Semantic corruption with a valid checksum:** unknown operation, invalid UTF-8, invalid delete value, or inconsistent internal lengths cause open to fail.
- **Absurd outer length:** parsing rejects it before attempting a huge allocation.
- **Bad or partial file header:** open fails clearly.
- **Injected short writes:** the write path completes them correctly or returns an error; it does not assume one write consumed every byte.
- **Injected append or sync failure:** no success acknowledgement and no further use of the handle until reopen.
- **Read error during recovery:** open returns an I/O error rather than treating it as EOF.
- **After tail repair, then append, then reopen:** the newly appended operation is present.
- **Compaction before replacement:** the original database remains authoritative.
- **Compaction after replacement:** the replacement has equivalent live state; subsequent writes go to the replacement file.
- **Leftover temporary file:** follow the documented authoritative-path policy.

For each test, record the initial bytes or state, fault location, expected result, and what evidence would disprove your claim.

Test organization

Use unit tests for validation and codecs. Use integration tests for actual filesystem behavior and CLI exit codes. Use subprocess tests for restart and termination boundaries. Test on disposable paths and avoid sharing a database between parallel tests.

Temporary directories make cleanup and isolation easier. Synchronization order can be inspected using a narrow test I/O abstraction and corroborated with system-call traces. Avoid a large mock framework that obscures the actual write path.

Before a checkpoint, run:

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test
```

A passing test suite is evidence about tested cases, not a proof of all possible failures. Keep the written invariants next to the tests that challenge them.

Useful optional extension

Property-based testing can generate more operation sequences and malformed inputs. Add it after the deterministic boundary tests are understandable. Preserve the seed or minimized failing input when a bug is found.

15. Linux debugging and profiling lab

Objective and setup

Observe how your storage decisions appear at the OS boundary. Perform this lab on a Linux VM or machine where the tools are available. A container may work, but profiling and tracing capabilities depend on host configuration. Record the actual environment and any unavailable tool rather than inventing a result.

On a Debian/Ubuntu-style system, tools commonly come from packages such as `build-essential`, `strace`, `gdb`, and `time`. `perf` packaging depends on the kernel and distribution. Install Rust separately through rustup.

Build the project in release mode and keep a debug build for debugger exercises. Record:

```
uname -a
rustc --version
cargo --version
cargo build --release
```

Exercise L1: trace a durable write

After implementing the CLI, use a disposable database path:

```
strace -f -o results/raw/put.trace \
  -e trace=openat,write,fsync,fdatasync,ftruncate,rename,renameat,renameat2,close \
  target/release/tiny-kv --db data/linux.kv put name Alice
```

Identify file creation/opening, record writes, synchronization, and close. Distinguish writes to the database from writes to stdout. Map the calls back to your implementation and check that success output follows the required synchronization call.

Trace compaction separately and identify temporary-file creation, file synchronization, rename, and directory synchronization. Exact syscall names can differ by library and platform.

Exercise L2: inspect a controlled failure

Use a debug build and `rust-gdb` to set a breakpoint in your record decoder or recovery loop. Open a deliberately malformed test database. Inspect the advertised payload length, last-valid offset, and error path. A backtrace should let you connect the CLI to the storage layer.

For a genuine panic while debugging, `RUST_BACKTRACE=1` can provide a stack trace. Expected invalid input should normally return an error rather than panic.

Submit a brief explanation of what you inspected and how it helped. A pasted stack trace without interpretation does not complete the exercise.

Exercise L3: CPU and memory evidence

Run your eventual workload binary directly under a CPU profiler. For example, `perf record -g -- <workload-command>` collects samples and `perf report` inspects them. Replace the placeholder with your actual reproducible command.

If unwinding or symbols are poor, build a dedicated profiling configuration with debug information and record any extra flags. Compare performance numbers from consistently configured binaries. On macOS, Instruments is an alternative for early profiling, but still complete the Linux syscall exercise.

Use GNU `/usr/bin/time -v <workload-command>` on Linux to record peak resident memory. On macOS, `/usr/bin/time -l <workload-command>` has different output and units. Label the tool and units in the report; resident memory and bytes allocated over time are different measurements.

Identify one hot path, explain why it consumes time, and connect it to Assignment 6's hypothesis. If storage waiting dominates, pair CPU evidence with elapsed time and synchronization observations.

16. Assessment rubric and submission checklist

Rubric: 100 points

Use this rubric for self-assessment or review by another engineer. Award points for evidence and explanation, not only code volume.

- **A1 - In-memory behavior and Rust foundations: 10 points.** Correct operations and validation (5), useful tests (3), ownership explanation (2).
- **A2 - Clean-restart persistence: 10 points.** Save/load behavior (5), format and I/O error handling (3), persistence/cost explanation (2).
- **A3 - Log and durability contract: 20 points.** Codec and replay (7), write/sync/ack ordering and initialization (8), uncertain-outcome explanation (5).
- **A4 - Recovery and failure tests: 20 points.** Boundary and corruption tests (8), correct repair/error policy (8), process-failure evidence and interpretation (4).
- **A5 - Compaction: 10 points.** Equivalent state and smaller output on the chosen dataset (4), safe replacement and handle management (4), boundary evidence (2).
- **A6 - Performance and Linux investigation: 15 points.** Reproducible baseline and profiles (5), controlled comparison (5), Linux tracing/debugging evidence (5).
- **A7 - Mini-LSM readiness: 5 points.** Readiness answers and Bitcask comparison (3), first-chapter evidence (2).
- **Engineering communication and reproducibility: 10 points.** Usable README and commands (3), design report and journal (4), clear commits and checkpoint evidence (3).

Total: 100 points. Optional extensions are enrichment rather than a substitute for missing core behavior.

Required correctness gates

Treat the core project as incomplete, regardless of point total, if any of these remain unresolved:

- Acknowledged mutations disappear in a tested failure covered by your contract.
- Opening a malformed database silently discards it and creates an empty one.
- Recovery accepts a broken tail and later appends behind it without repair.
- Compaction changes the visible live state or sends later writes to an obsolete file.
- The final documented demo cannot be reproduced from the repository.

Final submission checklist

1. Push the implementation, tests, documentation, and generated PDF to GitHub.
2. Ensure README commands describe the actual current implementation.
3. Commit `Cargo.lock` and document dependencies.

4. Complete `docs/progress.md` honestly.
5. Add `docs/design-report.md` following section 17.
6. Complete `results/performance-report.md` with actual observations.
7. Include small deterministic fixtures and enough commands to reproduce tests and measurements.
8. Keep generated databases, build outputs, and bulky raw profiles out of Git.
9. Run formatting, Clippy, and tests; record the final commit and environment.
10. Provide the Mini-LSM repository/checkpoint link and a final demonstration.

Suggested 8-10 minute demonstration

Show put/get/overwrite/delete across restarts. Explain the record layout using one small example. Demonstrate one interrupted-tail recovery and one hard corruption error. Compact an overwrite-heavy log. Present a before/after measurement and explain its limitations. Finish by answering when a write is safe to acknowledge.

17. Engineering workflow and design report

Commits and checkpoint tags

Make commits around understandable changes, such as `Implement missing-key behavior` or `Reject oversized records before allocation`. Avoid combining unrelated refactors with a recovery fix when it makes review harder.

At the completion of Assignment 1, for example:

```
git add src tests docs notes Cargo.toml Cargo.lock
git commit -m "Complete in-memory store checkpoint"
git tag -a a1-memory -m "Assignment 1 acceptance criteria complete"
git push origin main
git push origin a1-memory
```

Only include paths that exist at that point. Later checkpoint tags are listed in the assignment headings. Tags preserve educational stages even as later storage formats replace earlier ones.

Final design report outline

Create `docs/design-report.md` with these sections:

1. **Scope and assumptions:** supported platform, access model, data limits, and failure model.
2. **Architecture:** CLI, store, codec, filesystem interactions, and the in-memory representation.
3. **Read path:** how a get finds its value and what work happened during open.
4. **Write path:** validation, encoding, append, synchronization, map update, and acknowledgement.
5. **Disk format:** link to the specification and include one annotated byte example.
6. **Recovery:** valid boundaries, incomplete tails, hard errors, and repair ordering.
7. **Compaction:** replacement protocol, temporary-file policy, and active-handle transition.
8. **Tests:** representative failure scenarios, discovered bugs, and regression evidence.
9. **Performance:** the question, baseline, change, results, and tradeoffs.
10. **Limitations and next steps:** what the implementation does not establish and what you would investigate next.
11. **Sources and assistance:** references, reviewed examples, and significant AI assistance.

Aim for enough detail that another programmer could inspect your invariants and reproduce your evidence. A concise diagram or small byte trace often explains more than a long paragraph.

Example design-decision entry

Decision: keep all live values in the in-memory map.
Reason: make reads simple while learning the write and recovery paths.
Cost: memory scales with the entire live dataset; open must replay the log.
Evidence: measure peak resident memory for several dataset sizes.
Possible next design: index file offsets and read values from disk.

Review questions before moving on

Can I describe the change without reading the code aloud? Can I name its error paths? Does a test fail if I deliberately violate the intended behavior? Have I distinguished assumptions from observations? Are the README and status checklist accurate?

Store unresolved questions in the journal. They are useful inputs for the next session, especially when they concern an invariant rather than a syntax detail.

18. Your first two sessions

Session 1: get comfortable running Rust

The objective is a small successful learning loop.

1. Open the project directory in your editor.
2. Read the README and sections 1-4 of this handbook.
3. Install Rust using the official instructions if it is not present.
4. Run `rustc --version`, `cargo --version`, and `cargo run`.
5. Change one of the orientation messages in `src/main.rs` and run again.
6. Replace the orientation program with the small map example in Exercise P3.
7. Predict, then run, missing-key, overwrite, and delete examples.
8. Write your first short journal entry.

Expected observation: inserting `name = Alice` and retrieving it prints `Some("Alice")`; retrieving an absent key prints `None` with debug formatting.

If setup consumes the session, stop after getting the program to run and record the blocker or fix. That is real progress.

Session 2: ownership and a tiny API

1. Read the Rust Book's introductory ownership and borrowing sections.
2. Complete Exercise P2 and explain its compiler error.
3. Put the map into a `KvStore` struct.
4. Implement one operation and one meaningful test.
5. Run `cargo test` and deliberately check that the test catches a mistake.
6. Record what owns each string and where a borrow occurs.

Then continue through the remaining preparation exercises and Assignment 1. Files arrive in Assignment 2, once the in-memory behavior is clear.

When you get stuck

Capture:

```
Goal:
Command I ran:
Expected result:
```

```
Actual output or full error:
Relevant small code snippet:
My current explanation:
What I already tried:
```

This gives a tutor, reviewer, or coding assistant enough context to help you understand the next step. Preserve the first useful error message rather than making many speculative changes at once.

19. References and glossary

Primary learning resources

- **The Rust Programming Language:** https://doc.rust-lang.org/book/ - primary language reference; follow the targeted chapters in section 5.
- **Rust installation:** https://www.rust-lang.org/tools/install - current official toolchain installation instructions.
- **Rustlings:** https://github.com/rust-lang/rustlings - small exercises for language concepts that need more practice.
- **Rust standard library:** https://doc.rust-lang.org/std/ - look up `HashMap`, `File`, `OpenOptions`, `Read`, and `Write` as you use them.
- **File synchronization API:** https://doc.rust-lang.org/std/fs/struct.File.html#method.sync_all - read the guarantees and platform considerations of the method you call.
- **Mini-LSM:** https://skyzh.github.io/mini-lsm/ - next-stage storage-engine course, after Rust and log-store foundations.
- **Mini-LSM overview:** https://skyzh.github.io/mini-lsm/00-overview.html - bridge from an append-log store toward sorted tables and LSM architecture.
- **PingCAP Talent Plan:** https://github.com/pingcap/talent-plan/tree/master/courses/rust - additional Rust key-value-store project material.
- **Linux fsync documentation:** https://man7.org/linux/man-pages/man2/fsync.2.html - file and directory synchronization details.
- **Linux rename documentation:** https://man7.org/linux/man-pages/man2/rename.2.html - replacement semantics and filesystem constraints.
- **CRC implementation:** https://docs.rs/crc32fast/ - checksum dependency for the v1 record format.
- **Career context supplied for this project:** https://jobs.ashbyhq.com/turbopuffer/317f14bf-49dc-46ae-a4d8-6aea3fa41ecc - the original role link; postings may change or expire.

Glossary

- **Acknowledgement:** the database's success response to the caller.
- **Append-only log:** a sequence of records added at the end rather than updating old records in place.
- **Atomic operation:** an operation whose visible result is treated as one indivisible change in the relevant model; atomicity alone does not imply durability.
- **Checksum:** a compact value used to detect many forms of accidental byte corruption.
- **Compaction:** rewriting data to remove obsolete history or combine storage files.
- **Durability:** persistence of acknowledged data across failures covered by an explicit contract.
- **EOF:** end of file; distinguish a clean record boundary from an incomplete structure.
- **Invariant:** a condition the implementation must preserve, such as "only validated records affect recovered state."

- **LSM tree:** a storage organization that combines an in-memory write structure with immutable sorted tables and background merging.
- **Page cache:** operating-system memory used to cache file contents; bytes here are not automatically durable on storage.
- **Persistence:** data outliving a process; specify which failure scenarios are included.
- **Recovery:** reconstructing valid state from persistent data after opening or restarting.
- **Resident memory:** memory currently resident in physical RAM for a process, as defined by the measurement tool.
- **SST:** sorted string table; an immutable sorted on-disk table used by LSM-style engines.
- **Synchronization:** requesting that relevant buffered state be made persistent according to the OS and filesystem APIs.
- **Tombstone:** a stored deletion marker that prevents an older value from becoming visible again.
- **Torn write:** an interrupted write that leaves only part of the intended record.
- **Uncertain outcome:** a write may have taken effect even though the caller did not receive success.
- **Write amplification:** more bytes written by the storage system than the caller's logical update size.

Attribution

This is an original, AI-assisted learning assignment prepared for the tiny-kv project. It links to, rather than reproduces, the referenced courses. It is not affiliated with a university, Mini-LSM's author, or turbopuffer. The linked resources retain their respective licenses and authorship.